

LEZIONI 50

PERSISTENZA IN JAVA

JDBC E PERSISTENCY DATA ACCESS INTERFACES

Ingegneria del Software e Progettazione Web
Università degli Studi di Tor Vergata - Roma

Guglielmo De Angelis
guglielmo.deangelis@iasi.cnr.it

persistenza di oggetti – 1

(da Lezioni 41-42)

- il ciclo di vita “canonico” di un oggetto
 - inizia
 - l'allocazione di opportuno spazio di memoria (i.e. new in Java)
 - l'invocazione di uno dei costruttori definiti dalla classe di appartenenza
 - termina
 - la deallocazione esplicita dell'oggetto (i.e. in C++)
 - la deallocazione implicita da parte di un garbage collector (i.e. Java)
 - la terminazione della applicazione che ha creato l'oggetto

persistenza di oggetti – 2

(da Lezioni 41-42)

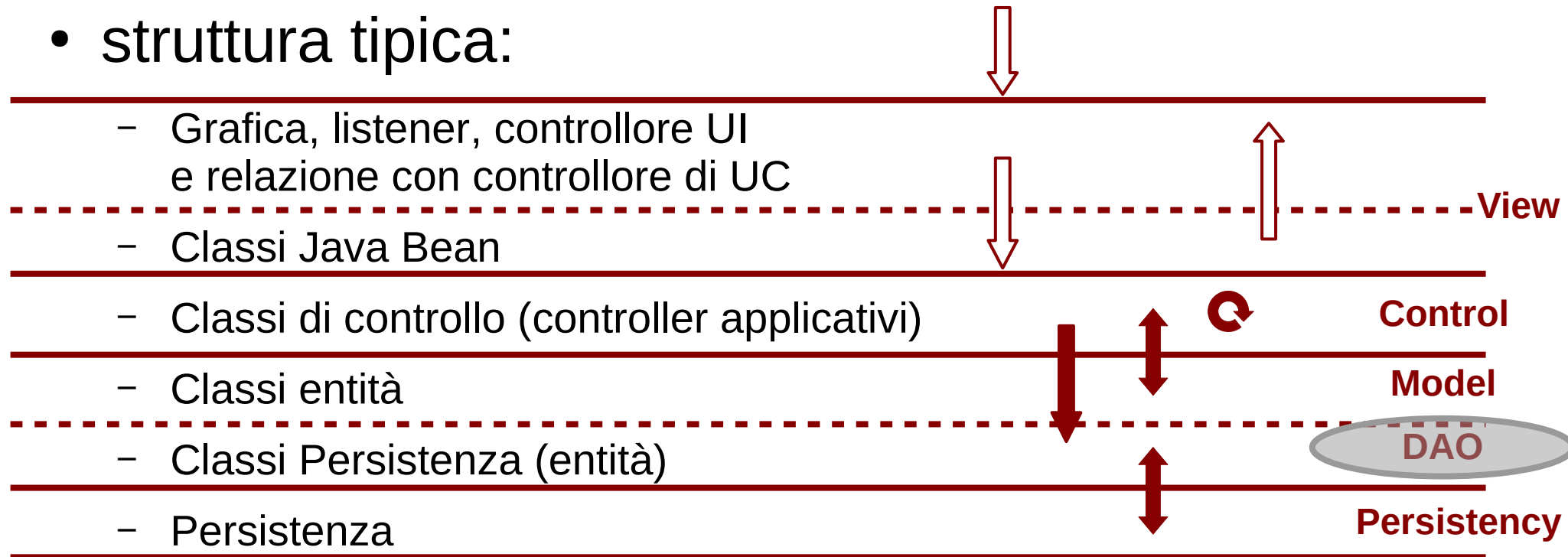
- il ciclo di vita “canonico” di un oggetto ha generalmente senso
- esistono molteplici scenari nei quali è comodo/utile/desiderabile che
 - lo stato di un oggetto continui ad esistere anche dopo la terminazione di una esecuzione dell'applicazione che lo ha creato
 - una applicazione possa configurare il suo contesto di esecuzione attraverso il ripristino dello stato di un insieme di oggetti
 - migrare l'esecuzione di un oggetto tra più nodi di una applicazioni distribuita

MVC in architetture laptop/stand-alone

- struttura tipica con persistency model

(da Lezioni 41-42)

- tutti i tre strati risiedono sulla macchina “utente”
- struttura tipica:



Interazioni lettura/scrittura mediate da classi bean



Interazioni lettura/scrittura SENZA la mediazione di classi bean

Classi DAO – 1

(da Lezioni 41-42)

- Data Access Object
- sono classi che offrono meccanismi di interfacciamento tra la rappresentazione O.O. di un dato e la corrispettiva rappresentazione in un qualche meccanismo di persistenza (e.g. DBMS relazionali, serializzazioni, file di testo o di markup)
- hanno la responsabilità di mediare tra la rappresentazione in memoria delle istanze (del model) e la loro rappresentazione su layer di persistenza
 - necessariamente rivestono anche la responsabilità di creator ed information expert per il tipo di dato trattato
- ad ogni classe nel model dovrebbe corrispondere un DAO che ne gestisce il salvataggio ed il recupero delle istanze su layer di persistenza

Classi DAO – 2

(da Lezioni 41-42)

schemi tipici di interazione:

- recupero delle entity da parte dei control
 - i control usano le informazioni nei beans per recuperare le classi model in memoria centrale
 - se una classe model non è in memoria centrale allora il control chiede al DAO (responsabile di quella classe model) che può
 - di recuperarla dalla persistenza
 - mantenere internamente una lista di istanze disponibili in memoria centrale e ritornala
- il DAO di una classe model può dover interagire con i DAO di altre classi model
- una classe model può far persistere i suoi dati per mezzo:
 - di una operazione che, invocata dal control, ne causa la storicizzazione su persistenza
 - del DAO che si registra come observer (vedi GoF Observer) dell'istanza model rilasciata (i.e. subject)
 - del control chiede esplicitamente al DAO di riportare in persistenza lo stato di una istanza in memoria centrale
- una classe model in memoria centrale può dover aggiornare il suo stato dalla persistenza per mezzo:
 - di una operazione che, invocata dal control, causa il refresh dei dati (l'istanza della classe model dovrà interagire con il relativo DAO)
 - del control che chiede esplicitamente al DAO di aggiornare lo stato di una istanza in memoria centrale
 - del DAO che periodicamente aggiorna i dati dell'istanza model rilasciata
 - del DAO che si registra come observer (vedi GoF Observer) degli eventi che avvengono nello strato di persistenza

persistenza di oggetti – 3

(da Lezioni 41-42)

- in generale l'uso di file o DBMS in parte sopperisce a queste necessità, tuttavia:
 - **file**: gestione esplicita delle convenzioni di memorizzazione e del formato dei dati utilizzato
 - **DBMS**: gestione e manutenzione generalmente non adatte a sistemi di dimensioni limitati o non specificatamente “enterprise”. Inoltre c'è da gestire il “mismatch” tra la rappresentazione Object Oriented del dominio e la rappresentazione specifica da adottare nel DB

persistenza di oggetti – 3

(da Lezioni 41-42)

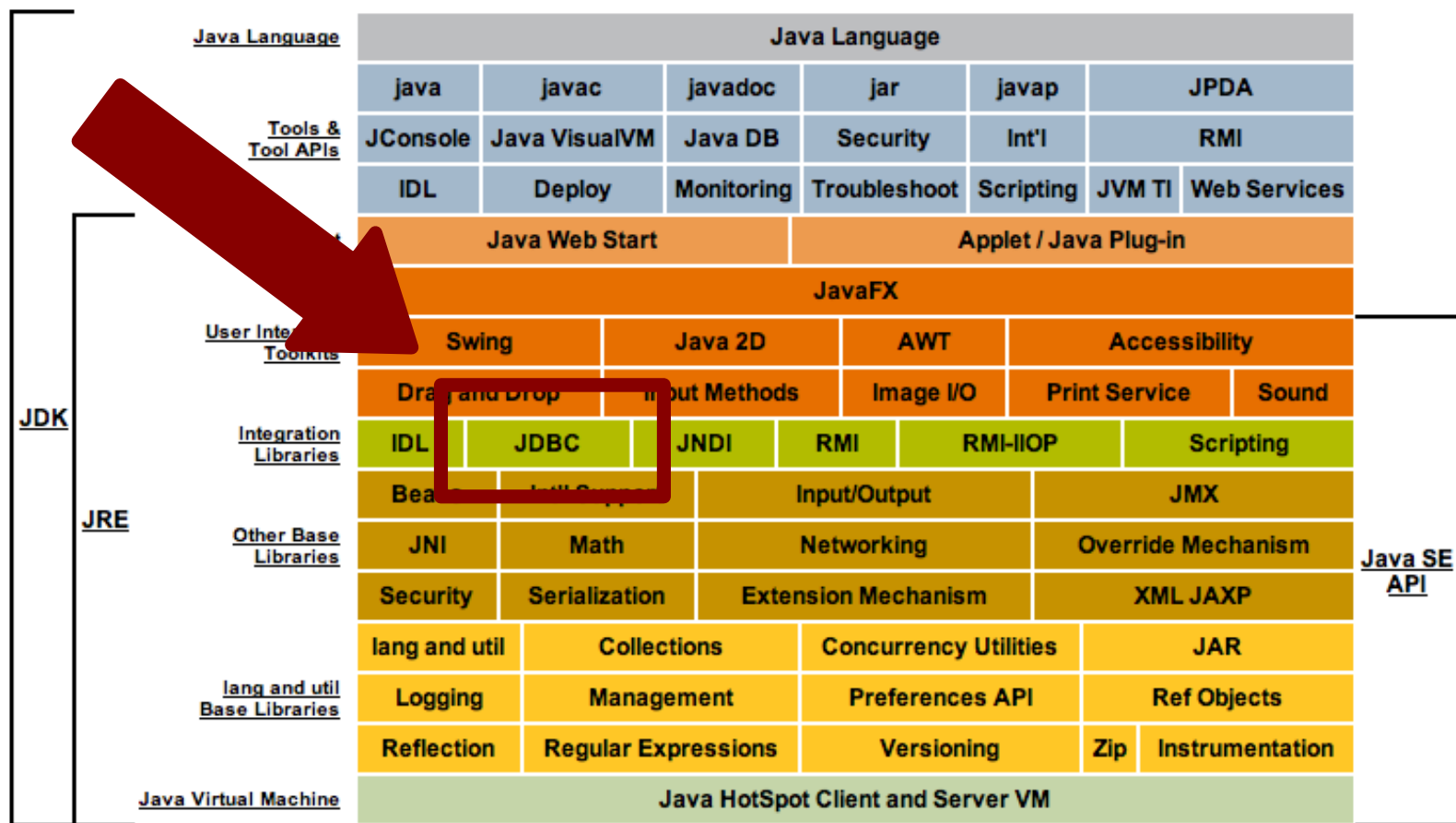
- in generale l'uso di file o DBMS in parte sopprime a queste necessità, tuttavia:
 - **file**: gestione esplicita delle convenzioni di memorizzazione e del formato dei dati utilizzato
 - **DBMS**: gestione e manutenzione generalmente non adatte a sistemi di dimensioni limitati o non specificatamente “enterprise”. Inoltre c'è da gestire il “mismatch” tra la rappresentazione Object Oriented del dominio e la rappresentazione specifica da adottare nel DB

JDBC – 1

- Java DB Connectivity framework
- framework per la connessione e l'interrogazione di DBMS relazionali
- consente di progettare applicazioni Java che siano agnostiche rispetto alla scelta di un DBMS
 - garantisce l'indipendenza di un'applicazione Java rispetto al deployment di differenti tecnologie (relazionali) per la persistenza
- ... tuttavia resta nelle responsabilità del programmatore la realizzazione di una completa astrazione rispetto al DBMS
 - nella interazione con JDBC utilizzare solo sintassi SQL standard (e.g. ANSI SQL 2)

JDBC – 2

- JDBC è parte integrante della piattaforma Java



- l'architettura di JDBC implementa concetti tipici della progettazione O.O.
 - astrazione delle funzionalità offerte attraverso una API comune: `java.sql.*`
 - binding su implementazioni alternative sulla stessa API
 - driver JDBC specifico per ogni DBMS relazionale
 - PostgreSQL JDBC Driver
 - JDBC Driver for MySQL (Connector/J)
 - HSQLDB JDBC Driver
 - UCanAccess an open-source JDBC Driver for Microsoft Access

macro classi per i driver JDBC

- Tipo 1: JDBC-ODBC Bridge
 - ODBC: Open DB Connectivity, legacy standard per l'interconnettività di DBMS.
 - le invocazioni JDBC sono mappate in ODBC
- Tipo 2: Driver Nativi
 - driver scritti in C, la loro invocazione richiede installazioni locali all'applicazione e dipendenti dalla piattaforma di esecuzione
- Tipo 3: Network Protocol Driver
 - scritti in Java, e con possibilità di essere installati remotamente all'applicazione
 - il driver ha la responsabilità di comportarsi da middleware verso uno o più DBMS
- Tipo 4: Altri Driver
 - scritti in Java e pensati per risiedere nella stessa macchina dove gira l'applicazione

una applicazione JDBC

- struttura tipica:

1. load a driver for a specific DBMS
2. connect to a DBMS
3. foreach query to the DB on DBMS {
4. instantiate an interaction **Statement**
5. query the DB
6. do something with the results
7. }
8. close the connection with the DBMS

una applicazione JDBC

- struttura tipica:

1. load a driver for a specific DBMS

2. connect to a DB

ATTENZIONE:

- E' UNA BUONA PRATICA MINIMIZZARE IL NUMERO DI CONNESSIONI APERTE CON IL DBMS
- SE POSSIBILE APRIRE UNA SOLA CONNESSIONE PER APPLICAZIONE PER CHIUDERLA SOLO QUANDO NON CI SARANNO ULTERIORI INTERAZIONI CON IL DBMS

6. do something with the results

7. }

8. close the connection with the DB

java.sql.Connection

- rappresenta una sessione di comunicazione con il DBMS
- è necessario aprirne una per leggere e scrivere sul DB
- l'apertura della connessione va richiesta al DriverManager specificando:
 - url del DBMS
 - nome del DB
 - username e password sul DBMS e con i permessi validi per il DB considerato

```
DriverManager.getConnection(  
    "jdbc:mysql://address/database_name",  
    "database_username", "database_password" );
```

java.sql.Statement

- astrae il concetto di operazione sul DB
 - deve essere usato per incapsulare il concetto di query SQL
- gli statement vanno istanziati a partire da una istanza di connessione attiva su un DBMS:

```
connection.createStatement()
```


java.sql.ResultSet

- modellano i dati in forma tabellare come estratti dal DB
- ResultSet offre l'operazione `next()` che scorre i record (relazionale) estratti dal DB
- per ogni record (relazionale), si può accedere ai singoli campi del DB
 - per indice della colonna (la numerazione comincia da 1)
 - per nome della colonna (come creato nel DB)
- i valori dei campi possono essere prelevati già convertiti nel tipo di interesse
 - `getString()`, `getDouble()`, `getInt()`,
 - `getBoolean()`...

```
String s = resultSetItem.getString(1); // preleva la prima  
colonna come stringa
```

```
int eta = resultSetItem.getInt("eta"); // preleva la  
colonna "età" come intero
```

riassumendo su entità e dati in persistenza

- entità
 - (nel contesto O.O.) sono classi che modellano le astrazioni chiave del dominio applicativo
 - le varie istanze sono relazionate tra loro attraverso riferimenti in memoria
- DAO (Data Access Object)
 - in una applicazione O.O. sono classi particolari che hanno il compito di mediare tra la rappresentazione in memoria delle istanze e la rappresentazione su layer di persistenza (e.g. DBMS relazionali)
 - ad ogni classe entità dovrebbe corrispondere un DAO che ne gestisce il salvataggio ed il recupero delle istanze su layer di persistenza

esempi JDBC

**VEDERE MATERIALE ALLEGATO
ALLA LEZIONE :**

`it.uniroma2.dicii.ispw.jdbcExamples.*`

esercizio JDBC

- Progettare versioni alternative per le classi DAO in:
 - `it.uniroma2.dicii.ispw.jdbcExamples.dao.*`
- che
 - evitino di stabilire ripetute connessioni con il DBMS
 - aprano connessioni solo se effettivamente non ne esistono di attive
 - condividano necessariamente lo stesso DB
- suggerimento: sfruttare altre classi e pattern GoF